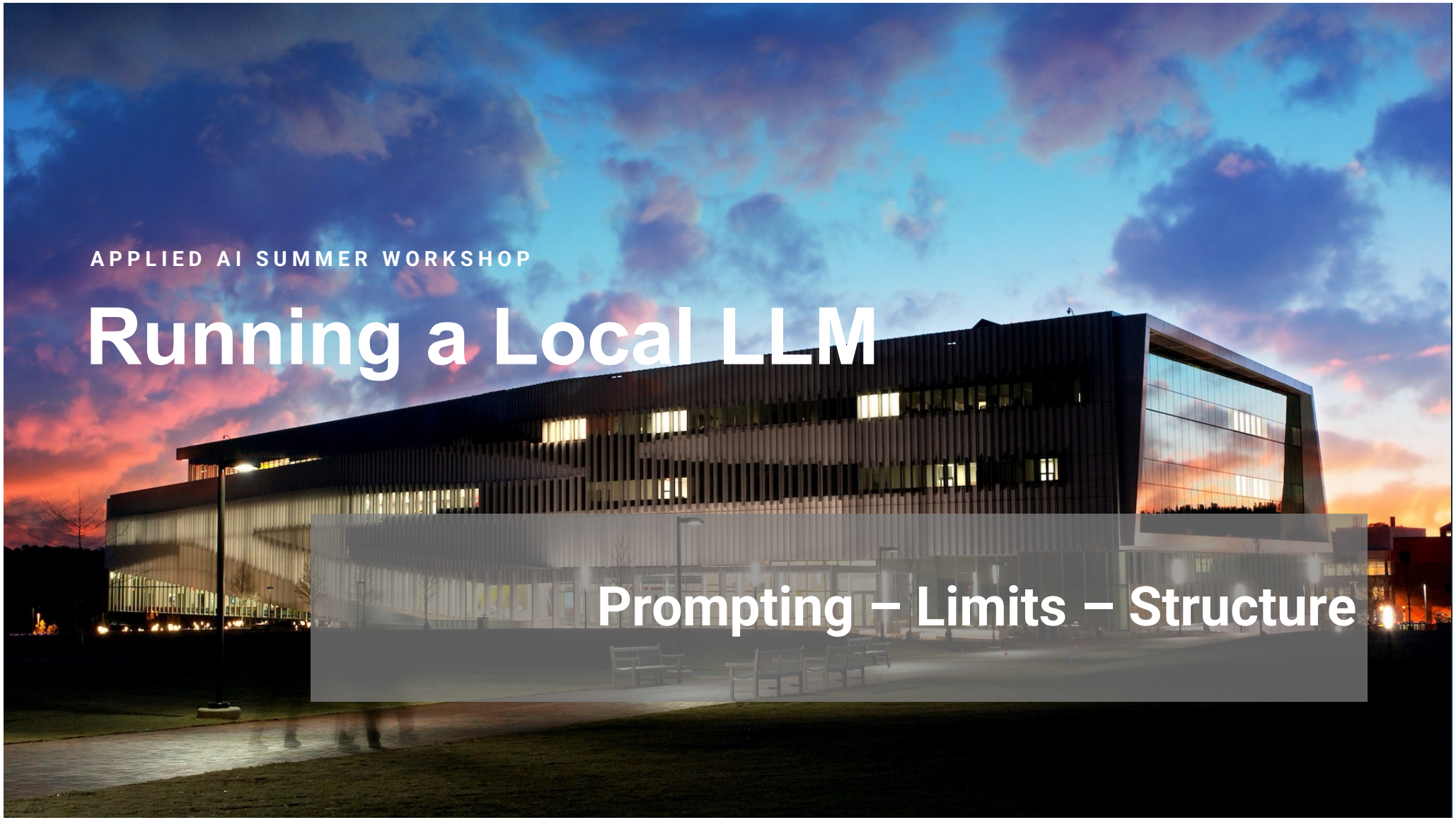

APPLIED AI SUMMER WORKSHOP

Running a Local LLM

Prompting – Limits – Structure

NC STATE Engineering

What this project is

A local language model you control end to end - no cloud, no account, nothing leaving the machine it runs on.

1	**Runs locally**	The model runs on the workshop JupyterHub or your own laptop via Ollama. No API key, no data sent to a vendor.
2	**No framework**	Every call is a raw HTTP POST to Ollama with the Python standard library - no LangChain, no SDK, nothing hidden.
3	**Readable top to bottom**	Each module is one flat file. You can read the whole pipeline in one sitting and see exactly what happens at each step.
4	**A small model, on purpose**	llama3.2:3b is ~3B parameters vs. 100B+ for cloud chatbots. Seeing where it breaks is part of the lesson.

NC STATE Engineering

Three ways to run the same material

Same modules, three delivery paths - pick whichever fits your setup on the day.

marimo notebooks

Click buttons, drag sliders, watch results update live.

NEEDS
Already installed on the NC State Hub

Private

Plain Python scripts

Read the code, edit values, re-run. No notebook software.

NEEDS
Standard library only (mostly)

Private

Google Colab

Run cells top to bottom in the browser; it installs everything itself.

NEEDS
Only a Google account needed

Not private - runs on Google's VM

Local model – Pros and Cons

NC STATE Engineering

+ Data privacy and IP control

+ No cost per token and no rate limits

+ Reproducibility and stability

- Not as capable

The whole integration is one function

NC STATE Engineering

One HTTP POST to /api/generate - a prompt goes in, text comes back. Everything in the workshop is built on this.

```
def call_ollama(prompt,
        system=None, options=None,
        response_format=None):

    payload = {
        "model": model,
        "prompt": prompt,
    }
    # + system / options / format
    r = POST("/api/generate", payload)
    return r["response"]
```

Four inputs, one per module

prompt	the question or task
system	how to behave -> Module 2
options	temperature, seed, ... -> Module 1
response_format	the shape of the reply -> Module 3

The setup

NC STATE Engineering

Log into the course Jupyter Hub
Clone the repository: Local_LLM_Demos
Go into Terminal from the main Hub window

```
cd Local_LLM_Demos
chmod a+x setup_ollama.sh
./setup_ollama.sh
```

This should install ollama and download some models we'll use for this demo
Try Ollama Out

```
bash
ollama list
ollama run llama3.2:3b
```

This will launch the llama3.2:3b parameter local model in your terminal. You can chat with the model here. Type `/quit` to leave the model.

You should now be able to use the Marimo demo scripts.

NC STATE Engineering

1

MODULE 1

Prompting & Parameters

The dials that control how the model generates text

MODULE 1 - PARAMETERS

NC STATE Engineering

temperature - the randomness dial

How much the model is allowed to gamble when picking each next word.

LOW - 0.0 - 0.3

Nearly deterministic. Picks the most likely word almost every time. Good for facts, code, repeatable answers.

HIGH - 0.8 - 1.5

Adventurous. Willing to pick less likely words - reads as more creative, and more prone to going off the rails.

LIVE DEMO - same prompt sent twice

```
"Write a one-sentence bedtime story about a sleepy professor."
```

At low temperature the two runs look nearly identical; crank it up and they drift apart. That divergence IS the randomness you dialed in.

MODULE 1 - PARAMETERS

NC STATE Engineering

top_p and top_k - narrowing the pool

Temperature decides how much to gamble; these decide which words are even on the table.

top_k	Keep only the k most likely words. top_k = 10 means 'only ever consider the 10 best candidates.' Lower = safer.
top_p	Keep the smallest set of top words whose probabilities add up to p (nucleus sampling). top_p = 0.9 covers 90% of the mass. Lower = safer.

LIVE DEMO - try top_k = 1 vs. top_k = 100

```
"Suggest a creative name for a campus coffee shop."
```

With top_k = 1 the model must take its single most likely word every time - output gets repetitive and 'safe.' Wider settings let variety through.

MODULE 1 - PARAMETERS

NC STATE Engineering

seed - making randomness repeatable

Same seed + same parameters + same prompt -> the same output, every run.

What it does

The seed fixes the starting point of the random number generator. Send the same prompt twice with the same seed and - even at high temperature - the two replies come back identical.

LIVE DEMO - sent twice with one seed

```
"Invent a name and one-line backstory
 for a time traveling professor."
```

Change the seed -> both runs change together.

?

'Why does it keep giving the professor the same name?'

Sampling parameters reshape a distribution the model already has - they can't invent variety it doesn't have. A prompt like this one puts almost all the probability on a handful of stock names. seed only picks which draw you take; temperature flattens the spike but rarely dethrones the leader. To get real variety: widen top_p / top_k, or change the prompt.

MODULE 1 - PARAMETERS

NC STATE Engineering

num_predict and stop - controlling length

Two dials that decide when the model stops talking.

num_predict

A hard cap on how many tokens the model may generate. Small values force short answers - and it's the main knob for keeping a slow local model responsive. The cap doesn't ask for brevity; it just cuts generation off.

stop

A list of strings that cut generation off the instant the model produces one. Handy for structured output - stop at a marker like 'END', or at '3.' to cut a numbered list short.

LIVE DEMO - set a small num_predict (say 16)

```
"List three tips for using AI effectively."   ->  watch the answer truncate mid-thought
```

MODULE 1 - RECAP

NC STATE Engineering

The five dials, at a glance

Everything in Module 1 is just building one options dictionary and passing it along.

temperature	how random / creative the output is
top_p, top_k	which candidate words are even considered
seed	makes random output repeatable
num_predict	hard cap on output length
stop	strings that cut generation off early

YOUR TURN

Build the options dict yourself so the answer is:

highly creative `-> temperature`

repeatable `-> seed`

capped short `-> num_predict`

Fill in the blanks, run twice, and confirm the seed makes it repeat.

NC STATE Engineering

2

MODULE 2

Prompts, Limits & Tools

The cheapest lever on behavior - and where a small model breaks

MODULE 2 - BEHAVIOR

NC STATE Engineering

System prompt vs. user prompt

Same question, different instructions for how to answer - the cheapest lever you have.

The user prompt is held fixed:

```
USER PROMPT (unchanged every run)
"Should I update my existing course notes or start from scratch?"
```

...only the system prompt preset changes:

Helpful assistant	Terse domain expert	Explain like I'm 5	Skeptical scientist	Pirate captain

Same model, same question - tone, structure, even the content of the answer shift. No retraining, just a different instruction string prepended to every request.

MODULE 2 - LIMITS

NC STATE Engineering

Where a small model breaks down

3B parameters is fast and private - but it fails in specific, learnable ways. Shown on purpose.

1

Letter counting

'How many r's in tangential?' - it sees tokens, not characters.

2

Multi-digit math

84,637 x 92,481 - pattern-matches instead of computing. Confident, wrong.

3

Knowledge cutoff

'Most recent World Cup?' - guesses from stale training data.

4

Multi-constraint

3 sentences, one starts with B, one has 7 words... drops a rule under load.

5

Hallucination

Summarize a novel that doesn't exist - invents a plausible plot.

These failures are fluent and confident, not obviously broken - which is exactly what makes them worth seeing live.

MODULE 2 - TOOLS

NC STATE Engineering

Basic tool use - give it a calculator

Don't ask the model to compute. Teach it to ask for a calculation, then run real code.

1 Ask

Send the question with a system prompt: reply with CALC: <expression> instead of guessing.

2 Check & calculate

If the reply starts with CALC:, pull out the expression and evaluate it for real in Python.

3 Answer again

Hand the exact result back to the model so its final answer uses real math, not a guess.

This is 'function calling' written out by hand. The same pattern extends to a web search, a database query, or document retrieval. That last one **is what RAG is** - a retrieval tool the model calls before answering, instead of a calculator.

NC STATE Engineering

3

MODULE 3

Structured Output

From 'hope it parses' to typed, validated data

MODULE 3 - FORMAT

NC STATE Engineering

A ladder to clean, machine-readable data

One new argument does all the work: response_format -> Ollama's format field.

1	**Naive prompt**	'...respond in JSON' and hope	you get **maybe-valid text**	*the baseline that breaks*
2	**format: 'json'**	Ollama guarantees valid JSON	you get **valid JSON, any shape**	*quick JSON, shape can vary*
3	**JSON Schema**	your fields, your types, required keys	you get **valid JSON, your shape**	*reliable field extraction*
4	**Pydantic**	one schema over many sentences	you get **a typed table**	*records you can compute on*

Each step fixes the one above it. Step 4 runs one Person schema over six differently-worded sentences and tabulates the results.

MODULE 3 - THE PAYOFF

NC STATE Engineering

Six messy sentences, one clean table

The same Person schema, one call per sentence - and results you can actually compute on.

EXTRACTED TABLE

#	name	age	city	occupation
1	Maria	34	Austin	software engineer
2	James Okonkwo	41	Seattle	cardiologist
3	Yuki Tanaka	29	Portland	chemistry teacher
4	Priya Raman	38	Chicago	accountant
5	Tom Alvarez	35	Miami	graphic designer
6	Ahmed Hassan	52	New York	architect

mean age 38.2 (range 29-52)

That average only works because age validated to a real int.

5 Age given as a birth year

"Born in 1990..." - the model has to subtract. The arithmetic weakness from Module 2, now hidden inside extraction.

6 No city in the text

city is a required str, so the model cannot answer "unknown". It invents one instead.

Structured output guarantees the SHAPE of an answer, not its TRUTH.
A schema makes results parseable, not correct - and a confidently wrong value in a clean table looks exactly like a right one. The fix: declare the field as `city: str | None = None` so the model can legitimately answer null instead of guessing.

NC STATE Engineering

THE LEVERS YOU NOW HAVE

What to take away

1 Prompt & parameters

The user prompt is the task; temperature, seed, and length dials shape how it's generated. Sampling can't create variety the model doesn't have.

2 System prompt

The cheapest behavior lever - tone, role, format - no retraining, just an instruction string.

3 Capability ceiling

No prompt fixes tokenization blindness or a knowledge cutoff. Some failures need tools; some need a bigger model.

4 Tools & structure

Delegate to real code for what the model can't do; constrain the output format so your code can parse it reliably.

Next up -> the Local LLM and RAG session: giving the model a document corpus it wasn't trained on.